Node.js Server Basics: Quick Theory Guide

For Beginners | Version 1.0 | January 2026 By Grok (xAI) *Hi Imdadul! This is a short, theory-only summary on basic Node.js servers. Copy-paste into Google Docs/Word, format as needed, and export to PDF for easy download.*

What is Node.js?

- **Runtime Environment**: Allows JavaScript to run on servers (not just browsers).
- **Core Engine**: Powered by Google's V8 (same as Chrome) for fast execution.
- **Key Strength**: Event-driven, non-blocking I/O – handles many requests efficiently without waiting.
- **Use Case**: Ideal for web servers, APIs, real-time apps (e.g., chat apps).

Why Learn It? JavaScript everywhere (front-end + back-end) = full-stack skills!

Building a Basic Server: Theory Overview

Core Concept: HTTP Module

- Node.js has a built-in http module for creating servers.
- **Steps in Theory**:
    1. **Import Module**: Load http to handle web requests.
    2. **Create Server**: Use createServer() – it takes a callback for incoming requests.
    3. **Request/Response**:
        - **req** (Request): Details like URL, method (GET/POST), headers.
        - **res** (Response): Send back data – status code, headers, body (e.g., HTML/text).
    4. **Listen**: Bind to a port (e.g., 3000) and host (e.g., localhost).
- **Run**: Execute with node filename.js – server starts, listens for browser requests.

Routing Basics

- **What?** Direct requests to different "pages" based on URL (e.g., /home vs /about).

- **How (Theory)**: Check req.url and req.method in the callback.
    - Match → Send specific response.
    - No match → 404 error.
- **Limits**: Manual checks get messy for big apps.

Intro to Frameworks (e.g., Express)

- **Why?** Simplifies routing, middleware (e.g., parsing JSON), error handling.
- **Theory Flow**:
    1. Install via npm: npm install express.
    2. Create app: const app = express();.
    3. Define routes: app.get('/path', callback).
    4. Listen: app.listen(port).
- **Benefits**: Less boilerplate, built-in tools for real-world servers.